# Joint UAV Placement and Dependent Task Offloading in Multi-UAV MEC Networks: A Graph Attention Enhanced DRL Approach

Cheng Zhan, *Member, IEEE*, Wei Liu, Kaifeng Song, Rongfei Fan, *Member, IEEE*, Jun Liu, *Member, IEEE*, and Han Hu, *Member, IEEE*

***Abstract*—Unmanned aerial vehicles (UAVs) have emerged as effective platforms for mobile edge computing (MEC), offering flexible and efficient computational support to ground users (GUs). Many practical applications, such as deep neural network inference tasks, generate subtasks with complex dependencies, significantly complicating scheduling and offloading decisions. In this paper, we study the joint optimization of UAV deployment, UAV-GU associations, and dependent task offloading decisions within a multi-UAV-enabled MEC system, aiming to minimize the end time of the overall tasks. The tasks generated by GUs are modeled using directed acyclic graphs (DAGs), explicitly capturing subtask dependencies and execution orders. To address the resulting complex optimization problem, we first propose a Joint Successive convex approximation and Penalty dual decomposition-based Optimization (JSPO) algorithm to determine the initial UAV deployment and UAV-GU associations. Next, we formulate the dependent task offloading decision process as a Markov decision process (MDP), which is solved by employing deep reinforcement learning (DRL). To effectively exploit the structural information within DAG tasks, we integrate a graph attention network (GAT) to provide enhanced state representations for DRL. JSPO and the DRL framework were executed in turns to gradually improve the performance. Extensive simulation results verify that our proposed framework significantly reduces the end time compared to existing methods, demonstrating its superiority in multi-UAV MEC systems.**

***Index Terms*—Mobile edge computing (MEC), unmanned aerial vehicle (UAV) deployment, dependent task offloading, deep reinforcement learning (DRL), graph attention network (GAT).**

Received 21 April 2025; revised 20 October 2025; accepted 29 October 2025. Date of publication 3 November 2025; date of current version 6 March 2026. This work was supported in part by the National Key Research and Development Program under Grant 2021YFC2803000, in part by the National Natural Science Foundation of China under Grant 62172339, Grant U2336211, and Grant 62431002, in part by the Fundamental Research Funds for the Central Universities under Grant SWU-KT25014, and in part by the Science and Technology Research Program of Chongqing Municipal Education Commission under Grant KJQN202500212. Recommended for acceptance by F. Wang. *(Corresponding author: Han Hu.)*

Cheng Zhan and Wei Liu are with the School of Computer and Information Science, Southwest University, Chongqing 400715, China (e-mail: zhanc@swu.edu.cn; auweliuw@email.swu.edu.cn).

Kaifeng Song and Han Hu are with the School of Information and Electronics, Beijing Institute of Technology, Beijing 100081, China (e-mail: 3120235659@bit.edu.cn; hhu@bit.edu.cn).

Rongfei Fan is with the School of Cyberspace Science and Technology, Beijing Institute of Technology, Beijing 100081, China (e-mail: fanrongfei@bit.edu.cn).

Jun Liu is with the School of Electronic and Information Engineering, Beihang University, Beijing 100191, China (e-mail: liujun2019@buaa.edu.cn).

Digital Object Identifier 10.1109/TMC.2025.3628608

## I. Introduction

With the rapid development of 5 G infrastructure and the Internet of Things (IoT), innovative mobile applications like augmented reality, autonomous driving, and smart healthcare systems have emerged [1]. These applications require fast, reliable connectivity and low-latency computing capabilities to provide seamless and real-time user experiences. However, many IoT devices are constrained by limited energy and computational resources, making it challenging to handle resource-intensive tasks locally. To address these challenges, mobile edge computing (MEC) has emerged as a promising solution [2], enabling devices to offload tasks to nearby edge servers, significantly reducing latency and alleviating the load on remote cloud infrastructure. Despite the advantages of MEC, its traditional static infrastructure faces significant limitations due to restricted coverage and fixed locations of edge servers. These geographical constraints can hinder service delivery, especially in remote or constrained environments. To overcome these limitations, unmanned aerial vehicle (UAV)-enabled MEC have been proposed as an innovative solution. UAVs offer flexible deployment, cost efficiency, and high mobility, allowing services to be deployed quickly to the areas where they are needed. Moreover, UAVs can establish direct line-of-sight (LoS) connections with ground users (GUs), enhancing connection stability and reliability while meeting quality-of-service (QoS) requirements [3].

In UAV-enabled MEC, resource-constrained GUs can offload computation-intensive tasks to UAV-mounted edge servers, alleviating their limitations in storage, battery life, and processing power. There are two offloading strategies commonly employed: partial offloading and binary offloading [4]. Partial offloading divides tasks into smaller components, allowing some to be processed locally and others to be offloaded to the UAV. However, partial offloading requires identifying meaningful partition points in the absence of clear computational boundaries, which is particularly challenging for complex tasks, and it increases synchronization costs between the GU and the UAV. On the other hand, binary offloading simplifies the decision-making process by requiring each task to be either entirely processed locally on the GU or completely offloaded to the UAV. This reduces complexity and synchronization overhead, offering better reliability. Therefore, we adopt binary offloading to simplify task management and enhance system reliability

1536-1233 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

and efficiency. To ensure efficient communication between GUs and UAVs, optimizing the UAV deployment and the UAV-GU association is essential. Many existing works [5], [6], [7] have studied this optimization problem, emphasizing its importance in enhancing communication efficiency. However, they often assume simplified tasks and overlook the real-world dependencies among tasks that can impact offloading and execution decisions.

Modern mobile applications, such as deep neural networks and augmented reality [1], increasingly rely on dependent tasks, where a task can only begin once its preceding tasks have completed. These dependencies are common in computation-intensive applications and pose significant challenges in UAV-enabled MEC systems. One of the key challenges is determining the optimal execution order for these tasks. Delays in one subtask can result in waiting times for subsequent tasks, leading to inefficient use of computational resources and increased overall tasks end time. Existing studies [8], [9], [10], [11] determine task scheduling order based on heuristic priorities. However, this experience-based and intuitive method often misses better solutions. Additionally, task offloading in UAV-enabled systems adds another layer of complexity. To minimize communication overhead and avoid delay from waiting for predecessor tasks, it is critical to optimize offloading decisions, especially when the system must coordinate between GUs and UAVs. Efficient collaboration between local devices and UAVs is essential for maximizing parallel computation and minimizing data transfers, which directly affects both computation and communication efficiency. Recent works [12], [13], [14], [15], [16], [17] have employed reinforcement learning to make offloading decisions, but heuristic methods are still commonly used for task scheduling. Furthermore, in the context of UAV-enabled MEC systems, the UAV deployments and UAV-GU associations significantly impact the system's communication environment. The location of UAVs and how they are associated with GUs directly affect data transfer rates and task offloading decisions, influencing both computation and communication efficiency. To ensure minimal the end time while considering task dependencies in UAV-enabled MEC systems, it is necessary to jointly optimize the UAV-GU association, UAV deployment positions, and task offloading decisions, which is challenging and has not been thoroughly investigated.

This paper investigates a multi-UAV-enabled MEC system for dependent tasks, where the UAV deployment, UAV-GU association, and task offloading are jointly optimized to minimize the end time for all tasks. The dependent tasks generated by GUs are modeled using a directed acyclic graph (DAG), with each node representing a subtask and the edges capturing the dependencies between them. The scheduling order and offloading decisions for each node are determined through deep reinforcement learning (DRL). To facilitate more effective learning from the DAGs, we integrate a graph attention network (GAT) into our framework. The GAT is trained in an unsupervised manner and acts as a plug-and-play module that provides a richer state representation for the DRL model, enhancing its ability to understand complex graph-based relationships. The main contributions of this paper are summarized as follows:

- We propose a comprehensive multi-UAV-enabled MEC framework in which each UAV hosts an aerial edge server to serve GUs generating dependent tasks. A joint optimization problem is formulated by encompassing UAV deployment, UAV-GU association, and offloading decision, with the objective of minimizing the maximum end time.
- We first simplify the original problem to establish an initial network configuration. Building on this, we develop an efficient iterative Joint Successive convex approximation and Penalty dual decomposition Optimization algorithm (JSPO), which iteratively refines UAV deployments and UAV-GU associations.
- The task offloading process is modeled as a Markov decision process (MDP) and solved using a DRL framework with dueling double deep Q-network (D3QN). We enhance our approach by combining the GAT with an iterative feedback loop between the JSPO algorithm and the DRL framework. The GAT fully exploits the DAG structure to provide a comprehensive state representation.
- Extensive simulations are conducted to validate the proposed approach. We analyze the task scheduling and offloading process, and investigate how bandwidth and task scale affect performance. The results show significant reductions in end time and improved resource utilization compared to existing benchmark methods.

## II. Related Work

### A. Task Offloading in UAV-Enabled MEC

Task offloading in MEC networks has become a key research area as the demand for efficient processing of computation-intensive and latency-sensitive tasks increases. In [18], computational-intensive tasks generated by Internet of Health Things (IoHT) devices were offloaded to edge servers to reduce latency and improve energy efficiency, while meeting ultra reliable and low latency communication (URLLC) requirements. In [19], the authors studied task computation delay minimization in multi-edge-server MEC systems, focusing on optimizing transmission duration and workload offloading allocation. They proposed a two-stage solution and a deep reinforcement learning approach for dynamic channels to efficiently determine offloading decisions. The work in [20] tackled the challenge of maximizing QoE of users in a MEC network by jointly optimizing resource allocation and task offloading decisions. In [21], the authors proposed a DRL framework where the actor network uses a DNN to model the relationship between input states and binary offloading decisions.

With the rising popularity of UAVs, UAV-enabled MEC systems have emerged as a flexible solution in MEC networks [3]. The work in [5] proposed an alternating optimization algorithm to reduce computing delays in UAV-assisted MEC systems, focusing on task scheduling and resource management. The authors in [6] addressed latency minimization in UAV-enabled MEC systems, with a focus on ultra-reliable and low-latency links between ground devices and a UAV-carried MEC server. Furthermore, the work in [7] developed a multi-UAV-enabled

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

MEC system to minimize energy consumption through optimized UAV deployment and task scheduling, employing a two-layer optimization method integrating differential evolution and greedy algorithms. While these studies highlight the benefits of task offloading and the integration of UAVs in MEC systems, they often overlook the real-world dependencies between tasks, which can significantly impact application performance and resource allocation.

### B. Offloading Dependent Tasks

Offloading tasks that have interdependencies poses unique challenges because the execution order and data transmission between subtasks must be carefully coordinated. Many existing studies have tackled this problem using heuristic algorithms [8], [9] or heuristic-based reinforcement learning [12], [13], [14]. The authors in [8] focused on designing a heuristic ranking-based algorithm to efficiently assign dependent tasks, aiming to minimize the average makespan. The authors in [9] proposed a heuristic centralized DNN dependent task offloading algorithm that addresses the fine-grained partition and scheduling of multiple DNN tasks. However, it relies on estimating computation loads and data transfers using intuitive priority metrics. In [12], the authors proposed a DRL method based on an actor-critic framework to minimize delay and energy consumption. In [13], the authors minimized average response time and introduce a mobility-aware dependent task offloading scheme using a DRL-based algorithm. They also proposed an intuitive priority determination algorithm and sorts tasks based on three key metrics. In [14], a deep deterministic policy gradient (DDPG)-based approach was proposed to optimize offloading policies, starting with a migration-enabled multi-priority task sequencing algorithm that determines the execution order by introducing several priority metrics to minimize deadline violations.

There are also studies that focus on UAV-enabled MEC systems for offloading dependent tasks. In [10], the joint problem of offloading linearly dependent tasks, designing UAV trajectories, and allocating resources was addressed by decomposing the problem into two subproblems using a block coordinate descent (BCD) method. In [11], a task reconfiguration algorithm transforms the task call graph into multiple sequential layers to simplify the dependency structure, though this linearization may miss more suitable scheduling sequences. In [15], the focus is on optimizing UAV trajectories and scheduling DAG tasks to reduce latency and energy consumption, yet the critical interdependencies between tasks are not fully considered. In [16], a UAV-assisted offloading scheme for vehicular networks minimized average task execution delay and energy consumption by introducing an asynchronous federated DRL-based algorithm. In [17], a double deep Q-network (DDQN)-based task offloading scheme was proposed in UAV-assisted smart farms, where a predictable metric is used to implement a priority-based scheduling approach, ensuring that tasks are executed in the correct order.

In summary, most existing approaches rely on heuristic methods for task prioritization and scheduling, often basing decisions on estimates or intuitive metrics that do not fully capture the complex interdependencies among subtasks. Different from the existing work, we design jointly optimization framework for UAV deployment, UAV-ground user associations, and dependent task offloading, where a graph attention enhanced DRL approach is developed to more effectively minimize the end time of the overall tasks and enhance overall system performance.

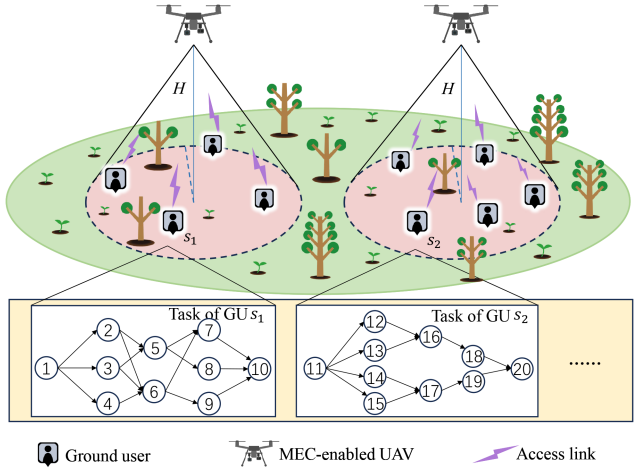



Fig. 1. Scenario illustration of the multi-UAV-enabled MEC system.

TABLE I
KEY NOTATIONS LIST

| Notation | Description |
|---|---|
| $\mathcal{M}, \mathcal{K}$ | Set of UAVs and set of GUs, respectively |
| $\mathbf{q}_m, \mathbf{w}_k$ | Horizontal coordinates of UAV $u_m$ and GU $s_k$ |
| $a_{k,m}$ | Association between GU $s_k$ and UAV $u_m$ |
| $G_k$ | DAG representing the task of GU $s_k$ |
| $c_i$ | Computational load (in cycles) of subtask $i$ |
| $d_{i,j}$ | Size of data transferred from subtask $i$ to subtask $j$ |
| $f_m, f_k$ | Computation capability of UAV $u_m$ and GU $s_k$ |
| $B$ | Signal bandwidth |
| $H$ | The UAV altitude |
| $P_k, P_m$ | The transmit power of GU $s_k$ and UAV $u_m$ |
| $TF$ | The final end time of all tasks |
| $\hat{R}_{k,m}$ | Uplink data rate from GU $s_k$ to UAV $u_m$ |
| $\check{R}_{m,k}$ | Downlink data rate from UAV $u_m$ to GU $s_k$ |
| $\mathcal{E}$ | The overall scheduling and offloading plan |
| $x_i$ | Binary offloading decision for subtask $i$ |
| $S(i), E(i)$ | Start time and execution time of subtask $i$ |
| $F(i)$ | End time of subtask $i$ |
| $T_n$ | The end time of the first $n$ scheduled subtasks |
| $TF$ | The final end time of all tasks |
| $\hat{d}_k$ | Data size to be uploaded for GU $s_k$ in special scenario |
| $\check{d}_k$ | Data size to be downloaded for GU $s_k$ in special scenario |

## III. SYSTEM MODEL AND PROBLEM FORMULATION

### A. Network Model

As shown in Fig. 1, we consider a multi-UAV-enabled MEC system where each UAV is equipped with an edge server that provides computing service to a set of $K$ GUs, which generate certain dependent tasks. Denote the set of GUs as $\mathcal{K} = \{s_1, s_2, \ldots, s_K\}$, where the horizontal coordinate of device $s_k \in \mathcal{K}$ is represented by $\mathbf{w}_k = [x_k, y_k]^{\mathrm{T}} \in \mathbb{R}^{2\times 1}$. For safety and simplicity considerations, we assume UAVs hover at a constant altitude with value $H$ [22]. The set of UAVs is denoted as $\mathcal{M} = \{u_1, u_2, \ldots, u_M\}$, where the horizontal position of

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

UAV $u_m \in \mathcal{M}$ is represented as $\mathbf{q}_m = [x_m, y_m]^{\mathrm{T}} \in \mathbb{R}^{2\times 1}$. Define UAV-GU association variables $\{a_{km}\}$, where $a_{km} \in \{0, 1\}$, $\forall k, m$. $a_{km} = 1$ if GU $s_k$ associates with UAV $u_m$; otherwise, $a_{km} = 0$. Similar to [16], we assume that each GU associates with only one UAV, i.e., $\sum_{m=1}^{M} a_{km} = 1, \forall k$, while a UAV can serve multiple GUs. The key notations used throughout this paper are summarized in Table I.

### B. Task Model

The computation task generated from $s_k$ can be represented as a DAG $G_k = (V_k, E_k)$, and the overall task graph $G$ is defined as $G = \bigcup_{k=1}^{K} G_k$. $V_k = \{v_{k,1}, v_{k,2}, \ldots, v_{k,|V_k|}\}$ is the set of nodes corresponding to the subtasks of $G_k$. $E_k$ denotes the set of dataflow edges. For an edge $(v_{k,p}, v_{k,q}) \in E_k$, the subtask $v_{k,p}$ is the predecessor of the subtask $v_{k,q}$. Thus, there are $N$ subtasks in total for all GUs, where $N = \sum_{k=1}^{K} |V_k|$.

To facilitate the discussion of the overall tasks generated by all GUs, each subtask is assigned an index $i$ in the range $1 \leq i \leq N$. The index $i$ uniquely identifies each subtask, allowing us to determine the GU $s_k$ and the specific subtask $v_{k,p}$. For each GU $s_k$, subtasks are indexed sequentially. If user $s_1$ has $|V_1|$ subtasks, subtasks for $s_2$ start from $|V_1| + 1$, and so on. The subtask with index $i$ corresponds to: $k = \min\{j \mid \sum_{k=1}^{j} |V_k| \geq i\}$ and $p = i - \sum_{k=1}^{k-1} |V_k|$. In simpler terms, $k$ is the smallest integer where the cumulative number of subtasks from 1 to $k$ is at least $i$, and $p$ is the position of the subtask within $V_k$ for the determined $k$. For example, consider the tasks in Fig. 1: If subtask 1 corresponds to $v_{1,1}$ generated by GU $s_1$, then subtask 11 corresponds to $v_{2,1}$ generated by GU $s_2$.

### C. Communication Model

Since UAVs are deployed at high altitudes, LoS connections can be established and dominate the communication channels [23]. Then, the channel gain between the UAV $u_m$ and GU $s_k$ is expressed as $h_{km} = \beta_0 d_{km}^{-\alpha}$, where $\alpha \geq 2$ is the path-loss exponent, $\beta_0$ is the channel power gain at the reference distance 1m, and $d_{km} = \sqrt{H^2 + \|\mathbf{q}_m - \mathbf{w}_k\|^2}$ is the distance between the GU $s_k$ and the UAV $u_m$. We assume that orthogonal channels are used for data transmission between GUs and their associated UAVs with no interference [24]. Subsequently, we delve into the details of the data transmission process.

*1) Ground-to-Air Links:* When a subtask is offloaded to the UAV, it involves transmitting dependency data from the predecessor subtasks that reside on the GU. The uplink data rate $\hat{R}_{k,m}$ from GU $s_k$ to UAV $u_m$ is given by

$$\hat{R}_{k,m} = B\log_2\left(1 + \frac{\gamma_k}{(H^2 + \|\mathbf{q}_m - \mathbf{w}_k\|^2)^{\alpha/2}}\right), \quad (1)$$

where $B$ denotes the communication bandwidth, and $\gamma_k \triangleq \frac{P_k \beta_0}{\sigma^2}$ represents the received signal-to-noise ratio (SNR) at 1m. Here $P_k$ is the transmit power of $s_k$, and $\sigma^2$ denotes the background noise power.

*2) Air-to-Ground Links:* After UAV executes, each subtask outputs the necessary data for its successor subtasks. Downloading occurs when the subtask on the GU needs dependency data from its predecessor subtask on the UAV. The downlink data rate $\check{R}_{m,k}$ from the UAV $u_m$ to GU $s_k$ is given by

$$\check{R}_{m,k} = B\log_2\left(1 + \frac{\gamma_m}{(H^2 + \|\mathbf{q}_m - \mathbf{w}_k\|^2)^{\alpha/2}}\right), \quad (2)$$

where $\gamma_m \triangleq \frac{P_m \beta_0}{\sigma^2}$, and $P_m$ is the transmission power of the UAV $u_m$.

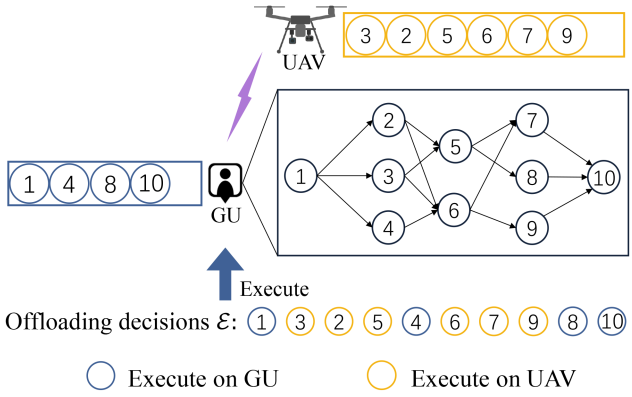



Fig. 2. An example of how a single task is offloaded.

### D. Task Scheduling Model

For the overall tasks with $N$ subtasks, the scheduling plan, denoted as $\mathcal{E}$, is a permutation of the set $\{(i, x_i)\}_{i=1}^{N}$, where each tuple $(i, x_i)$ indicates a scheduling decision for subtask $i$ and its position and the permutation sequence determines the sequence of scheduling. Specifically, $x_i = 1$ means the subtask $i$ is offloaded to the associated UAV, while $x_i = 0$ means it is executed locally. As illustrated in Fig. 2, consider an scheduling plan sequence where: (1) the system first chooses decision (1, 0). Subtask 1 is dispatched to the GU's local queue; (2) the system then chooses (3, 1). Subtask 3 is dispatched to the UAV's queue; (3) the system chooses (2, 1), offloading subtask 2 to the UAV's queue;...;(10) the decision (10, 0) is chosen, and subtask indexed 10 is executed locally. The scheduling plan can be represented as $\mathcal{E} = \{(1, 0), (3, 1), (2, 1), \ldots, (10, 0)\}$.

According to the scheduling plan, the tasks of all GUs execute based on $\mathcal{E}$. Moreover, define $T_n$ as the maximum end time of the first $n$ scheduled subtasks. Then, the end time of all tasks is denoted as $TF$, which is the maximum end time of all $N$ subtasks, and it is defined as

$$TF = T_N. \quad (3)$$

### E. Latency Calculation Model

Since subtasks are executed on different devices, $T_n$ is defined as the end time of the first $n$ scheduled subtasks. Specifically, $T_n$ is expressed as

$$T_n = \max_{i \in \mathcal{D}_n}\{F(i)\}, \quad (4)$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

where $\mathcal{D}_n$ is the set of the first $n$ scheduled subtasks. $F(i)$ is the end time of the subtask $i$, which is defined as

$$F(i) = S(i) + E(i), \tag{5}$$

where $S(i)$ and $E(i)$ represent the start and execution time of the subtask $i$, respectively.

The execution time $E(i)$ depends on the computation amount of the subtask $i$ and the computation capability of the executing device, defined as

$$E(i) = \frac{(1-x_i)c_i}{f_k} + \sum_{m=1}^{M} a_{km}\frac{x_i c_i}{f_m}, \tag{6}$$

where $c_i$ denotes the calculation amount for executing the subtask $i$. $f_k$ and $f_m$ represent the computation capabilities of the GU $s_k$ and the UAV $u_m$, respectively. It functions as a mathematical switch, where the first term applies if the subtask is processed locally ($x_i = 0$) and the second term applies if it is offloaded ($x_i = 1$).

The start time $S(i)$ is influenced by two key factors. The first is the arrival of results from all predecessors, which determines the data available time. The second is the availability of the assigned computing resource, which determines the resource available time. It is defined as

$$S(i) = \max\left\{\max_{j\in pre(i)}\{F(j) + H(i,j)\}, I(i)\right\}, \tag{7}$$

where $pre(i)$ denotes the set of predecessor subtasks of subtask $i$. The first term, $\max_{j\in pre(i)}\{F(j) + H(i,j)\}$, represents the data available time. This is the time when all necessary data from every predecessor subtask has arrived. The second term, $I(i)$, represents the resource available time, which is the time when the assigned processor becomes available. $H(i,j)$ is the transmission latency of dependency data from subtask $i$ to subtask $j$, given by

$$H(i,j) = \frac{(1-x_i)x_j d_{i,j}}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} + \frac{x_i(1-x_j)d_{i,j}}{\sum_{m=1}^{M} a_{km}\check{R}_{m,k}}, \tag{8}$$

where $d_{i,j}$ is the size of the data transfer from subtask $i$ to subtask $j$. The first term calculates the uplink latency, while the second calculates the downlink latency. If both subtasks are on the same device, both terms become zero. Additionally, $I(i)$ is the time when the device that executes subtask $i$ becomes idle, which is defined as

$$I(i) = (1-x_i)I_k + x_i\sum_{m=1}^{M} a_{km} I_m. \tag{9}$$

Here, $I_k$ is the CPU idle time of the GU $s_k$, who generates subtask $i$, and $I_m$ is the idle time of UAV $u_m$. Both $I_k$ and $I_m$ are updated after each subtask is completed.

For example, consider the start time $S(6)$ for subtask 6 in Fig. 2, which is executed on the UAV. Its predecessors, subtasks 2 and 3, are also on the UAV, while subtask 4 is on the GU. For subtask 6 to start, two conditions must be met. First, all predecessor data must be ready. This means waiting for subtasks 2 and 3 to end, and for subtask 4 to end and have its data transmitted to the UAV. The moment the last of these data dependencies is fulfilled determines the data available time. Second, the UAV's own processor must be free, which determines the resource available time.

### F. Problem Formulation

This paper aims to minimize the end time $TF$ for tasks of all GUs by jointly optimizing UAV deployment $\{\mathbf{q}_m\}$, the user association $\{a_{km}\}$, and offloading plan $\mathcal{E}$. The problem is formulated as

$$\begin{aligned}
(\text{P1}): \quad & \min_{\{\mathbf{q}_m\},\{a_{km}\},\mathcal{E}} \quad TF \\
\text{s.t.} \quad & a_{km} \in \{0,1\}, \forall k, m, && (10)\\
& \sum_{m=1}^{M} a_{km} = 1, \forall k, && (11)\\
& x_i \in \{0,1\}, 1 \le i \le N, && (12)\\
& x_l = 0, l \in \mathcal{L}, && (13)\\
& S(i) \ge F(j), \forall j \in pre(i), && (14)\\
& TF \ge F(i), \forall i \in 1,\ldots,N, && (15)
\end{aligned}$$

where $\mathcal{L}$ is the set of start and end subtasks in all tasks. Constraint (13) indicates that these subtasks are executed locally due to user-interaction and device I/O tasks [25], such as capturing initial input or displaying a final result. Such operations are inherently tied to the GU and cannot be offloaded. Constraint (14) ensures the subtask is executed only after all its predecessor subtask have finished. Constraint (15) ensures that the overall task is completed only when all subtasks are finished. Problem (P1) is extremely complex due to the constraint relationships between subtasks and device heterogeneity. Moreover, the problem has several binary decision variables, which makes it challenging and prevents finding the exact optimal solution directly. Specifically, optimizing a task order requires evaluating all possible topological sortings of a DAG with $n$ subtasks and $e$ edges, resulting in a factorial time complexity of $O((n+e)\cdot n!)$ [26]. Additionally, offloading each task to one of $K$ heterogeneous devices introduces an exponential complexity of $O(K^n)$. Even when the task execution order and offloading is predetermined, the resulting problem remains a non-convex optimization, which is difficult to solve.

To make the problem more tractable, we first focus on a simplified but practical scenario in which all intermediate subtasks are offloaded to UAVs, while the start and end subtasks are executed locally on GU devices due to (13). This assumption enables a tractable two-stage decomposition of the problem. It allows us to decouple the complex joint optimization by first solving a more manageable subproblem: finding the foundational UAV deployment and user association based primarily on the communication costs associated with the offloaded intermediate tasks. This allows us to establish a stable and efficient network configuration using the JSPO algorithm. Furthermore, although we fix the scheduling of start and end subtasks to achieve tractability, this choice reflects real deployment constraints in UAV-enabled MEC. In typical use cases such as interactive

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

sensing or actuation, initial data collection and final response must occur locally to avoid latency and reliability issues. The heavy computation stages in between are offloaded to UAVs that are prepositioned to ensure coverage and maintain reliable LoS connections. By capturing these considerations, our scenario remains tractable and clearly applicable to practical operations.

## IV. UAV Deployment and User Association Under Special Scenario

### A. Special Scenario

Consider a special scenario where the subtask scheduling is predefined: intermediate subtasks are offloaded to UAVs, while the start and end subtasks are processed locally on GU devices. This approach simplifies the problem by reflecting practical constraints. Start and end subtasks typically require user interaction or immediate I/O operations, making local execution necessary to minimize latency and handle data effectively [9]. Meanwhile, offloading intermediate subtasks to UAVs capitalizes on their high-performance computing resources, reducing the end time of the overall tasks, particularly when GUs have limited computational capabilities. Additionally, this scenario mirrors real-world conditions, such as event-driven deployments where UAV networks are preconfigured to optimize coverage and connectivity.

Based on this setup, the problem (P1) is reformulated to minimize the end time $TF'$ of overall tasks in this special scenario by jointly optimizing of UAV deployment and user association. For a given GU task $G_k$, the end time of the task, $T_k$, can be expressed as

$$T_k = T_k^{\text{loc}} + T_k^{\text{tran}} + T_k^{\text{UAV}}, \tag{16}$$

where $T_k^{\text{loc}}$ represents the time required to execute the start and end subtasks locally, and $T_k^{\text{UAV}}$ denotes the time for UAVs to execute all intermediate subtasks of task $G_k$. $T_k^{\text{tran}}$ represents the time to upload the data generated by the start subtask and download the data required by the end subtask, which is given by

$$T_k^{\text{tran}} = \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{R}_{k,m}}, \tag{17}$$

where $\hat{d}_k = \sum_{j\in suc(v_{k,1})} d_{1,j}$ and $\check{d}_k = \sum_{j\in pre(v_{k,|V_k|})} d_{j,|V_k|}$, representing the sizes of the data to be uploaded and downloaded, respectively. $suc(v_{k,1})$ is the set of successor subtasks of subtask $v_{k,1}$.

Since the execute time on UAVs $T_k^{\text{UAV}}$ is a constant value, it can be ignored in the such scenario. Moreover, $T_k^{\text{loc}}$ is not affected by the optimization variables. Therefore, the problem reduces to minimizing $T_k^{\text{tran}}$, which can be reformulated as

$$\begin{aligned}(\text{P2}): \quad \min_{\{\mathbf{q}_m\},\{a_{km}\}} \max_{s_k\in\mathcal{K}} & \left\{ \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} \right. \\ & \left. + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{R}_{k,m}} \right\} \\ \text{s.t.} \quad & (10), (11).\end{aligned}$$

Note that the objective function in (P2) involves binary variables, complex rate expressions, and coupled variables. Thus (P1) is a mixed-integer non-convex optimization problem, which is difficult to solve directly. To address this challenge, we first decompose the problem (P2) into two sub-problems and propose corresponding solutions, i.e., SCA-based UAV deployment and PDD-based user association optimization. We then propose JSPO algorithm, where the two subproblems are alternately optimized until convergence is achieved.

### B. UAV Deployment Optimization

In this subsection, we focus on optimizing UAV deployment $\{\mathbf{q}_m\}$ with given user association $\{a_{km}\}$. The objective function in (P2) is non-convex and difficult to solve directly due to the rate expressions in the denominator. To make this problem tractable, the strategy is to reformulate it into a more structured form. We begin by introducing slack variables $\{\hat{y}_{k,m}\}$ and $\{\check{y}_{k,m}\}$ to represent the uplink and downlink rates, and an overall variable $\Upsilon$ to represent the maximum transmission time. This allows us to separate the complex terms from the objective, leading to the following, more manageable problem formulation:

$$\begin{aligned}(\text{P3}): \quad \min_{\Upsilon,\{\mathbf{q}_m\}} \quad & \Upsilon \\ \text{s.t.} \quad & \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{y}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{y}_{k,m}} \le \Upsilon, \forall k, m, \end{aligned} \tag{18}$$

$$\hat{R}_{k,m} \ge \hat{y}_{k,m}, \forall k, m, \tag{19}$$

$$\check{R}_{k,m} \ge \check{y}_{k,m}, \forall k, m. \tag{20}$$

It can be shown that the optimal solution to (P3) occurs when equality holds in constraints (19) and (20), which can be proven by contradiction. However, (P3) remains a non-convex optimization problem due to the non-convex constraints in (19) and (20). To tackle the non-convex constraints in (P3), we adopt the SCA method [27], which is an iterative optimization technique. The key idea of the SCA is to replace the non-convex components of the optimization problem with convex approximations, making it possible to solve the problem iteratively [28].

Note that $\hat{R}_{k,m}$ and $\check{R}_{k,m}$ are convex functions with respect to $\|\mathbf{q}_m - \mathbf{w}_k\|^2$. Using a first-order Taylor expansion around the current local point $\mathbf{q}_m^r$ at the $r$-th iteration, we derive a lower bound for the uplink rate $\hat{R}_{km}$, i.e.,

$$\begin{aligned}\hat{R}_{k,m} &\ge -\hat{\mathcal{A}}(\|\mathbf{q}_m - \mathbf{w}_k\|^2 - \|\mathbf{q}_m^r - \mathbf{w}_k\|^2) + \hat{\mathcal{B}} \\ &\triangleq \hat{R}_{k,m}^{lb},\end{aligned} \tag{21}$$

where $\hat{\mathcal{A}} \triangleq \frac{B\gamma_k(\alpha/2)\log_2 e}{\Omega(\Omega^{\alpha/2}+\gamma_k)}$, $\hat{\mathcal{B}} \triangleq B\log_2\left(1+\frac{\gamma_k}{\Omega^{\alpha/2}}\right)$, and $\Omega \triangleq H^2 + \|\mathbf{q}_m^r - \mathbf{w}_k\|^2$. Similarly, the lower bound for the downlink rate $\check{R}_{km}$ is derived as

$$\begin{aligned}\check{R}_{k,m} &\ge -\check{\mathcal{A}}(\|\mathbf{q}_m - \mathbf{w}_k\|^2 - \|\mathbf{q}_m^r - \mathbf{w}_k\|^2) + \check{\mathcal{B}} \\ &\triangleq \check{R}_{k,m}^{lb},\end{aligned} \tag{22}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

where $\check{\mathcal{A}} \triangleq \frac{B\gamma_m(\alpha/2)\log_2 e}{\Omega(\Omega^{\alpha/2}+\gamma_m)}$, $\check{\mathcal{B}} \triangleq B\log_2\left(1+\frac{\gamma_m}{\Omega^{\alpha/2}}\right)$. Substituting these lower bounds into problem (P3), the approximation problem becomes:

$$(\text{P4}): \min_{\Upsilon,\{\mathbf{q}_m\}} \quad \Upsilon$$

$$\text{s.t.} \quad \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{y}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{y}_{k,m}} \le \Upsilon, \forall k, m, \tag{23}$$

$$\hat{R}^{lb}_{k,m} \ge \hat{y}_{k,m}, \forall k, m, \tag{24}$$

$$\check{R}^{lb}_{k,m} \ge \check{y}_{k,m}, \forall k, m. \tag{25}$$

Problem (P4) is a convex optimization problem and can be solved using CVX [29]. Furthermore, by substituting $\hat{R}_{km}, \check{R}_{km}$ with their respective lower bounds, any feasible solution to (P4) will also be feasible for (P3). Thus, the objective value of (P4) provides an upper bound for the objective value of (P3), ensuring that the solutions obtained from (P4) serve as valid approximations to the optimal solution of (P3).

### C. User Association Optimization

In this subsection, given the UAV deployment $\{\mathbf{q}_m\}$, we focus on optimizing the user association $\{a_{km}\}$, which transforms problem (P1) into problem (P5):

$$(\text{P5}): \min_{\{a_{km}\}} \max_{s_k\in\mathcal{K}} \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{R}_{k,m}}$$

$$\text{s.t.} \quad (10), (11).$$

To address the challenges posed by the binary constraints in (10) and (11), we first relax this integer constraint, allowing $\{a_{km}\}$ to be a continuous variable. We achieve this by introducing a continuous auxiliary variable $\{\hat{a}_{km}\}$ and then adding constraints that force $\{\hat{a}_{km}\}$ to ultimately take on binary values (0 or 1) at the optimal solution. This reformulation allows us to apply powerful continuous optimization tools. The equivalent set of tractable algebraic conditions is suitable for continuous optimization methods:

$$0 \le \hat{a}_{km} \le 1, \tag{26}$$

$$\sum_{m=1}^{M} \boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k = 1,\ \boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k = \hat{a}_{km},\ \boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1) = 0, \tag{27}$$

where $\boldsymbol{a}_m \triangleq [a_{1m},\ldots,a_{Km}]^{\mathrm{T}}$, and $\mathbf{e}_k$ is the $k$-th column of the identity matrix $\mathbf{I}_K$. With these auxiliary variables, problem (P5) is now equivalent to:

$$(\text{P6}): \min_{\{a_{km}\},\{\hat{a}_{km}\}} \max_{s_k\in\mathcal{K}} \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{R}_{k,m}}$$

$$\text{s.t.} \quad (26), (27).$$

In (P6), the variables $a_{km}$ are no longer restricted to be binary. To solve this, we develop a PDD-based algorithm, which uses a double-loop structure. The PDD framework [30] is employed incorporating a penalized version of the equality constraints in (27) and introducing a slack variable $\eta$ into the objective function, and then we obtain the following augmented Lagrangian (AL) problem:

$$\begin{aligned}(\text{P7}): \min_{\{a_{km}\},\{\hat{a}_{km}\},\eta} \quad & \eta + \frac{1}{2\rho}\sum_{k=1}^{K}\sum_{m=1}^{M} |\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k - \hat{a}_{km} + \rho\tilde{\lambda}_{km}|^2 \\ & + \frac{1}{2\rho}\sum_{k=1}^{K}\sum_{m=1}^{M} |\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1) + \rho\bar{\lambda}_{km}|^2 \\ & + \frac{1}{2\rho}\sum_{k=1}^{K}\left|\sum_{m=1}^{M}\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k - 1 + \rho\hat{\lambda}_k\right|^2 \\ \text{s.t.} \quad & 0 \le \hat{a}_{km} \le 1, \qquad (28) \\ & \frac{\hat{d}_k}{\sum_{m=1}^{M} a_{km}\hat{R}_{k,m}} + \frac{\check{d}_k}{\sum_{m=1}^{M} a_{km}\check{R}_{k,m}} \le \eta, \qquad (29)\end{aligned}$$

where $\{\tilde{\lambda}_{km}\}, \{\bar{\lambda}_{km}\}, \{\hat{\lambda}_k\}$ are the Lagrange multipliers, and $\rho \in \mathbb{R}_+$ denotes the penalty coefficient. Problems (P6) and (P7) are equivalent as $\rho \to 0$, which is a key characteristic of the PDD framework. In the PDD-based algorithm, the inner loop solves the AL problem with fixed multipliers and penalty coefficient, while the outer loop updates the dual variables and reduces the penalty coefficient based on constraint violations. The optimization of the inner-loop variables $\{a_{km}\}$, $\{\hat{a}_{km}\}$ and $\eta$ described as follows.

1) *Optimization of* $\{\hat{a}_{km}\}$*:* The optimization of $\{\hat{a}_{km}\}$ is performed while fixing the other variables. The corresponding subproblem for $\{\hat{a}_{km}\}$ can be written as:

$$\begin{aligned}(\text{P8}): \min_{\{\hat{a}_{km}\}} \quad & \frac{1}{2\rho}\left(|\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k - \hat{a}_{km} + \rho\tilde{\lambda}_{km}|^2\right. \\ & \left. + |\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1) + \rho\bar{\lambda}_{k\iota}|^2\right) \\ \text{s.t.} \quad & 0 \le \hat{a}_{km} \le 1. \end{aligned} \tag{30}$$

Problem (P8) presents a scalar quadratic objective function in terms of $\hat{a}_{km}$. The unconstrained minimizer of the objective function can be directly determined from the first-order optimality condition:

$$\hat{a}'_{km} = \frac{\rho\tilde{\lambda}_{km} + a_{km} - \rho a_{km}\bar{\lambda}_{km} + a_{km}^2}{1 + a_{km}^2}. \tag{31}$$

Given the constraint $0 \le \hat{a}_{km} \le 1$, the optimal solution for the constrained problem (P8) is obtained as follows:

$$\hat{a}^*_{km} = \begin{cases} 0, & \hat{a}'_{km} \le 0, \\ \hat{a}'_{km}, & 0 < \hat{a}'_{km} < 1, \\ 1, & \hat{a}'_{km} \ge 1. \end{cases} \quad \forall k, m. \tag{32}$$

2) *Optimization of* $\eta$ *and* $\{a_{km}\}$*:* The subproblem for $\eta$ and $\{a_{km}\}$ can be expressed as:

$$(\text{P9}): \min_{\{a_{km}\},\eta} \quad \eta + \frac{1}{2\rho}\sum_{k=1}^{K}\sum_{m=1}^{M} |\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k - \hat{a}_{km} + \rho\tilde{\lambda}_{km}|^2$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 1:** JSPO Algorithm for Problem (P2).

Initialize $\{\mathbf{q}_m^0\}, \{a_{km}^0\}, \{\hat{a}_{km}^0\}$ and dual variables $\{\tilde{\lambda}_{km}, \bar{\lambda}_{km}, \hat{\lambda}_k\}^0$; Set $r \leftarrow 0$, $t \leftarrow 0$;
**repeat**
  Given $\{a_{km}^r\}$ and $\{\mathbf{q}_m^r\}$, solve (P4) using CVX solver to obtain the optimal solution $\{\mathbf{q}_m^{r+1}\}$;
  **repeat**
    **repeat**
      Update $\hat{a}_{km}$ according to (32);
      Update $a_{km}^r$ by solving (P9) by CVX;
    **until** The relative decrease in the objective value of (P7) is less than $\epsilon_1$;
    Calculate the constraint violation $\delta$ according to (37);
    **if** $\delta \leq \epsilon^t$ **then**
      Update dual variables according to (34)–(36);
    **else**
      Update $\rho^{t+1} = \tau\rho^t$;
    **end if**
    Update $\epsilon^{t+1} \leftarrow \tau\delta$, and $t \leftarrow t+1$;
  **until** The constraint violation $\delta$ less than $\epsilon_2$;
  Update $a_{km}^{r+1} \leftarrow a_{km}^r$, and $r \leftarrow r+1$;
**until** The relative decrease in the objective value of (P2) is less than $\epsilon_3$.

$$+\frac{1}{2\rho}\sum_{k=1}^{K}\sum_{m=1}^{M}|\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1)+\rho\bar{\lambda}_{km}|^2$$
$$+\frac{1}{2\rho}\sum_{k=1}^{K}\left|\sum_{m=1}^{M}\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k-1+\rho\hat{\lambda}_k\right|^2$$
$$\text{s.t.}\quad \frac{\hat{d}_k}{\sum_{m=1}^{M}a_{km}\hat{R}_{k,m}}+\frac{\check{d}_k}{\sum_{m=1}^{M}a_{km}\check{R}_{k,m}}\leq\eta. \tag{33}$$

The objective function in (P9) includes both linear and quadratic terms, while the constraint (33) remains convex. As a result, (P9) is a convex optimization problem and can be effectively solved using the CVX toolbox [29].

During the outer iteration of the PDD-based algorithm, the dual variables $\{\tilde{\lambda}_{km}\}$, $\{\bar{\lambda}_{km}\}$ and $\{\hat{\lambda}_k\}$ are updated according to

$$\tilde{\lambda}_{km}^{t+1}=\tilde{\lambda}_{km}^{t}+(\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k-\hat{a}_{km})/\rho^t, \tag{34}$$
$$\bar{\lambda}_{km}^{t+1}=\bar{\lambda}_{km}^{t}+(\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1))/\rho^t, \tag{35}$$
$$\hat{\lambda}_{k}^{t+1}=\hat{\lambda}_{k}^{t}+\left(\sum_{m=1}^{M}\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k-1\right)/\rho^t. \tag{36}$$

Here, $t$ denotes the iteration number of the outer loop. The penalty parameter $\rho$ is updated as $\rho^{t+1}=\tau\rho^t$, where $0<\tau<1$. The constraint violation indicator $\delta$ is used to measure the extent of equality constraint violations, and it is defined as

$$\delta=\max_{k,m}\left\{|\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k-\hat{a}_{km}|,|\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k(\hat{a}_{km}-1)|,\left|\sum_{m=1}^{M}\boldsymbol{a}_m^{\mathrm{T}}\mathbf{e}_k-1\right|\right\}. \tag{37}$$

The PDD-based algorithm for user association optimization terminates when $\delta\leq\epsilon_2$, where $\epsilon_2$ is the tolerance for constraint violation. Thus, the iterative solution process for UAV deployment and user association is outlined in Algorithm 1. Note that the overall complexity is dominated by solving the convex problem (P3) and (P9), which have complexities of $\mathcal{O}(M^{3.5})$ and $\mathcal{O}((MK)^{3.5})$, respectively. Thus, the total computational complexity of the Algorithm 1 is given by $\mathcal{O}(I_1I_2I_3M^{3.5}(MK)^{3.5})$, where $I_1$, $I_2$ and $I_3$ are the number of iterations with the given solution accuracy $\epsilon_1$, $\epsilon_2$ and $\epsilon_3$, respectively.

## V. Adaptive GAT for Task Offloading With D3QN

In this section, we first introduce the design of the GAT to encode the DAG structure of tasks into high-dimensional representations. Then, given the UAV positioning and user association derived in Section IV, the D3QN-based DRL scheduler is proposed to efficiently identify the optimal offloading decisions in dynamic and complex environments. Finally, the complete GAT-enhanced D3QN offloading (GATD3QN) algorithm is presented.

### A. GAT Encoder Design

Task scheduling for DAGs is particularly challenging due to the inherent dependencies and parallelism of the graph structure. Effective scheduling requires capturing the characteristics of the graph structure and node information. GAT [31], [32] as a specific type of graph neural network (GNN) offers significant advantages in graph-based tasks. GAT leverages an attention mechanism to aggregate the features of a node's neighbors, updating its own features based on the graph structure captured by the adjacency matrix. Moreover, as the number of experience samples increases, GAT becomes more effective and generalizable. While GAT is effective at feature extraction from graph data, it comes with a higher computational cost due to its larger parameter set compared to conventional neural networks. To address this challenge, we propose pretraining a GAT encoder separately in an unsupervised learning. This pretraining enables the encoder to extract graph features, which are then translated into state vectors for further exploit in the following DRL. In this context, the GAT encodes a DAG of dependent tasks, where each node corresponds to a subtask. By extracting both node features and structural dependencies, the GAT enables a clear understanding of the subtasks and their dependencies, providing essential insights for the DRL model.

The GAT encoder consists of multiple stacked graph attention encoder layers. The input to the encoder is a set of node features, represented as

$$\boldsymbol{h}=\{\mathbf{h}_1,\mathbf{h}_2,\ldots,\mathbf{h}_N\},\mathbf{h}_i\in\mathbb{R}^F, \tag{38}$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

where $N$ is the number of nodes of the graph, $\mathbf{h}_i$ is the feature vector of node $i$, and $F$ is the number of features for each node. Each layer of encoder generates new representations of nodes by utilizing the information from their neighbors. The relevance between nodes and their neighbors is calculated using a self-attention mechanism, where the attention weights are shared across nodes [31]. Specifically, in the $l$-th encoder layer, the relevance of a neighboring node $j$ to node $i$ is computed as

$$e_{ij}^{(l)} = \text{Sigmoid}(\text{ReLU}(\mathbf{a}^{(l)^{\mathrm{T}}}[\mathbf{W}^{(l)}\mathbf{h}_i^{(l-1)}||\mathbf{W}^{(l)}\mathbf{h}_j^{(l-1)}])), \tag{39}$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{F^{(l)} \times F^{(l-1)}}$ and $\mathbf{a}^{(l)} \in \mathbb{R}^{F^{(l)}}$ are the trainable parameters of the $l$-th layer, ReLU denotes the leaky ReLU activation function, $||$ represents the concatenation of two matrices, and Sigmoid is used for squashing the attention score. Next, we normalize the attention scores by using the softmax function:

$$\alpha_{ij}^{(l)} = \frac{\exp\left(e_{ij}^{(l)}\right)}{\sum_{k \in \mathcal{N}_i} \exp\left(e_{ik}^{(l)}\right)}, \tag{40}$$

where $\mathcal{N}_i$ represents the neighborhood of node $i$ according to the adjacency matrix $\mathbf{A}$.

Using the initial node features $\boldsymbol{h}$, the representation of node $i$ in layer $l$ is updated as

$$\mathbf{h}_i^{(l)} = \sum_{j \in \mathcal{N}_i} \alpha_{ij}^{(l)} \text{ReLU}\left(\mathbf{W}^{(l)}\mathbf{h}_j^{(l-1)}\right). \tag{41}$$

After passing through all $L$ layers, the final representation of the overall task graph $G$ is obtained as

$$\hat{\boldsymbol{h}} = \boldsymbol{h}^{(L)} = \{\hat{\mathbf{h}}_1, \hat{\mathbf{h}}_2, \ldots, \hat{\mathbf{h}}_N\}, \hat{\mathbf{h}}_i \in \mathbb{R}^{F^{(L)}}. \tag{42}$$

To ensure that the learned graph representations are effective, we evaluate the quality of the output representations. To this end, we use a decoder with a similar architecture to reverse the encoding process in an unsupervised manner. Specifically, each decoder layer reconstructs the node representations by leveraging the representations of neighboring nodes based on their relevance. The input to the decoder is $\hat{\boldsymbol{h}}$,, and the output of the $l$-th decoder layer is $\hat{\boldsymbol{h}}^{(l)}$. The final output of the decoder is obtain as

$$\check{\boldsymbol{h}} = \hat{\boldsymbol{h}}^{(L)} = \{\check{\mathbf{h}}_1, \check{\mathbf{h}}_2, \ldots, \check{\mathbf{h}}_N\}, \check{\mathbf{h}}_i \in \mathbb{R}^{F}. \tag{43}$$

We minimize the reconstruction loss of the node features to ensure effective feature extraction, i.e.,

$$\text{Loss}_1 = \sum_{i=1}^{N} \|\mathbf{h}_i - \check{\mathbf{h}}_i\|_2. \tag{44}$$

Additionally, even if there is no direct edge between two nodes, their representations should be similar if their features are similar. Therefore, we minimize the reconstruction loss of the graph structure by ensuring that the representations of neighboring nodes are close:

$$\text{Loss}_2 = -\sum_{i=1}^{N} \sum_{j \in \mathcal{N}_i} \log\left(\frac{1}{1 + \exp(-\hat{\mathbf{h}}_i^{\mathrm{T}}\hat{\mathbf{h}}_j)}\right). \tag{45}$$

TABLE II
EXAMPLE OF THE SCHEDULING AND OFFLOADING PROCESS WITH 10 TASKS

| $t$ | $o_t$ | $T_t$ | $a_t$ |
|---|---|---|---|
| 0 | $(-1,-1,-1,-1,-1,...,-1)$ | 0 s | (1,0) |
| 1 | $(0,-1,-1,-1,-1,...,-1)$ | 0.124 s | (3,1) |
| 2 | $(0,-1,1,-1,-1,...,-1)$ | 0.215 s | (2,1) |
| 3 | $(0,1,1,-1,-1,...,-1)$ | 0.292 s | (5,1) |
| 4 | $(0,1,1,-1,1,...,-1)$ | 0.292 s | (4,0) |
| ... | ... | ... | ... |
| $t$ | $o_t$ | $T_t$ | $a_t$ |
| ... | ... | ... | ... |
| 10 | $(0,1,1,0,1,...,0)$ | 0.684 s | - |

The total loss function combines both the feature and structure reconstruction losses:

$$\text{Loss} = \text{Loss}_1 + \lambda \text{Loss}_2, \tag{46}$$

where $\lambda$ is a regularization parameter.

### B. DRL Design

In this work, we proposed a D3QN-based task offloading method, which enables efficient identification of the optimal offloading strategy in complex environments. The D3QN algorithm separates the estimation of state values and state-dependent action advantages, which are then intelligently combined through an aggregation layer to compute the action value function [33]. Compared to standard deep Q-network (DQN), the dueling architecture allows for more efficient learning of state value functions, while also being more robust to approximation errors, especially when the action value gaps between different actions in the same state are small [33], [34]. Additionally, double Q-learning has been widely adopted to address the overestimation bias caused by the maximum operation, utilizing two separate networks to handle action selection and value evaluation during bootstrapping [35].

Table II illustrates a scheduling and offloading process involving 10 subtasks, as shown in Fig. 2. To index the decision process, we introduce $t$, where $t \in [0, N]$. While $i$ indexes the subtasks, $t$ represents the decision step at which the decision for each subtask $i$ is made. The state of all subtasks is represented as a vector $(x_1, x_2, \ldots, x_N)$, where $x_i$ equals $-1$ for unoffloaded subtasks, 0 for locally executed subtasks, and 1 for offloaded subtasks. Initially, all subtasks are unoffloaded, so the state vector is $(-1,-1,-1,\ldots,-1)$. For example, after scheduling and executing subtask 1 locally, the state updates to $(0,-1,-1,-1,-1,\ldots,-1)$ with the execution time $T_1 = 0.124$ s. When the subtask 3 is offloaded to the UAV for execution, the state becomes $(0,-1,1,-1,-1,\ldots,-1)$, and the execution time updates to $T_2 = 0.215$ s. This step-by-step process continues until all subtasks have been scheduled. Finally, when the state $o_{10} = (0,1,1,0,1,\ldots,0)$ is reached, all subtasks are executed, and the end time of all subtasks is calculated as $TF = T_{10} = 0.684$ s. Note that subtasks in the DAG are executed in parallel (e.g., $T_3 = T_4 = 0.292$ s), meaning that the end time $T_t$ corresponds to the maximum end time among the scheduled subtasks.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 2:** Calculation of $T_t$.

**Input:** The $t$-th scheduling decision $(i, x_i)$ in $\mathcal{E}$;
Denote the executing device of subtask $i$ as $d$ based on $x_i$;
**if** $t = 0$ **then**
  Initialize $\mathcal{D}_0 = \emptyset$, and $I_k = I_m = 0, \forall k, m$;
**else**
  Obtain $\mathcal{D}_{t-1}$and $I(i)$ according to (9);
**end if**
**for** each subtask $j$ in $pre(i)$ **do**
  Get the assigned device of subtask $j$ as $d'$ based on $x_j$;
  **if** $d = d'$ **then**
    $H(i, j) = 0$;
  **else**
    Calculate $H(i, j)$ according to (8);
  **end if**
**end for**
Calculate $S(i), E(i)$ and $F(i)$ according to (7), (6), and (5);
**if** $x_i = 0$ **then**
  Update $I_k = F(i)$;
**else**
  Get the associated UAV $u_m$ based on $a_{km}$;
  Update $I_m = F(i)$;
**end if**
Update $\mathcal{D}_t = \mathcal{D}_{t-1} \cup (i, x_i)$;
**Output:** Calculate $T_t$ according to (4).

To solve the task offloading problem, we model the system as a MDP, aiming to maximize the cumulative rewards. The corresponding state space, action space, and reward function are defined as follows.

*State Space:* State is the input of DRL and typically represents the system model, guiding the DRL agent's actions. However, in this work, the system model includes a graph structure with both node-level and graph-level attributes, making it difficult to regularize the data. The GAT encoder addresses this by integrating graph structure attributes into node-level attributes, allowing a set of node attributes to represent the overall task state. We refer to the raw attributes as observations and the final DRL input as state, which is in a deeper feature representation and ultimately leads to a more accurate state-value function. We define the observation of tasks as

$$\mathcal{O}_t = \{o_t, \mathcal{O}_{\text{node}}, \mathbf{A}\}, \tag{47}$$

where $o_t$ indicates the state of all subtasks at decision step $t$ and $\mathbf{A}$ is the adjacency matrix. $\mathcal{O}_{\text{node}}$ is a set of attributes of subtasks, which consists of four attributes: 1) $c_i$: the calculation amount for executing the subtask $i$; 2) $\sum_{j \in suc(i)} d_{i,j}$: data size generated by subtask $i$; 3) in-degree; 4) out-degree.

Then, the state space is denoted as $\mathcal{S}$, where $s_t \in \mathcal{S}$ represents the state at decision step $t$ and is obtained by

$$s_t = \text{GATEncoder}(\mathcal{O}_t). \tag{48}$$

*Action Space:* The action space is denoted as $\mathcal{A} = \{(1, 0), (1, 1), (2, 0), (2, 1), \ldots, (N, 0), (N, 1)\}$, where an action $a_t \in \mathcal{A}$ represents the offloading decision at decision step $t$. The next observation of all subtasks $o_{t+1}$ is generated after executing action $a_t$ at observation $o_t$. Specifically, if $a_t = (i, 1)$, the subtask $i$ is offloaded, changing $x_i$ from $-1$ to 1in $o_{t+1}$. As illustrated in Table II, after executing $a_1 = (3, 1)$ at $o_1 = (0, -1, -1, -1, -1, \ldots, -1)$, the observation is transferred to $o_2 = (0, -1, 1, -1, -1, \ldots, -1)$. Then the next state $s_{t+1}$ is obtained by (48). Once all actions are generated, a complete scheduling plan is formed. Note that certain scheduling operations are not permitted under specific conditions. Specifically, the following constraints must be met: (1) A subtask cannot be offloaded to the UAV if it is a start or end subtask that needs to be executed locally; (2) A subtask cannot be rescheduled once it has been executed; (3) Due to task dependencies, a subtask can only be executed after all its predecessors have been completed. To satisfy the above constraints, we use a node mask vector as in Fig. 3, which is determined based on the adjacency matrix $\mathbf{A}$ and the subtasks states $o_t$ to filter out unavailable actions according to the three constrains mentioned above. The computation of the mask vector is based on the static task dependency graph represented by the adjacency matrix A, the dynamic state of all subtasks at the current step $t$ represented by the observation vector $o_t$ and the three constraints. This approach also ensures that the output dimension of the D3QN remains unchanged. After an action interacts with the environment, the node mask vector is updated along with the observation. The mask is updated at each step by checking two conditions for every subtask: whether it has already been scheduled, and whether all its predecessors are complete. For example, consider the task in Fig. 2. Initially, only subtask 1 meets these criteria, so the mask vector, which indicates task availability, would be [True, False, False, . . .], restricting the agent's first action. Once subtask 1 is scheduled, its position is set to False. Now, its successors, subtasks 2, 3, and 4, have their dependencies met and are unscheduled, so their corresponding positions in the mask are flipped to True. The updated mask becomes [False, True, True, True, False, . . .]. This process repeats at each step, dynamically pruning the action space and ensuring every selected action is valid according to the task dependencies.

*Reward function:* The reward function guides the D3QN agent to learn the optimal offloading plan to minimize overall tasks end time. At each decision step $t$, the agent makes an offloading decision based on the current state. The reward function is designed to directly guide the agent toward minimizing the total task completion time. At each decision step $t$, the agent's action $a_t$ updates the total accumulated latency to $T_{t+1}$. We define the immediate reward $r_t$ as the negative change in this latency:

$$r_t = T_t - T_{t+1}, \tag{49}$$

where $r_t$ is the reward after executing $a_t$ at $s_t$, $T_t$ represents the accumulated latency at decision step $t$, and $T_{t+1}$ represents the accumulated latency at decision step $t + 1$. Thus, $T_t - T_{t+1}$ is non-positive. The absolute value of the reward represents the increase in latency due to the agent's decision. Thus, the agent's goal of maximizing the reward translates directly to making this

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

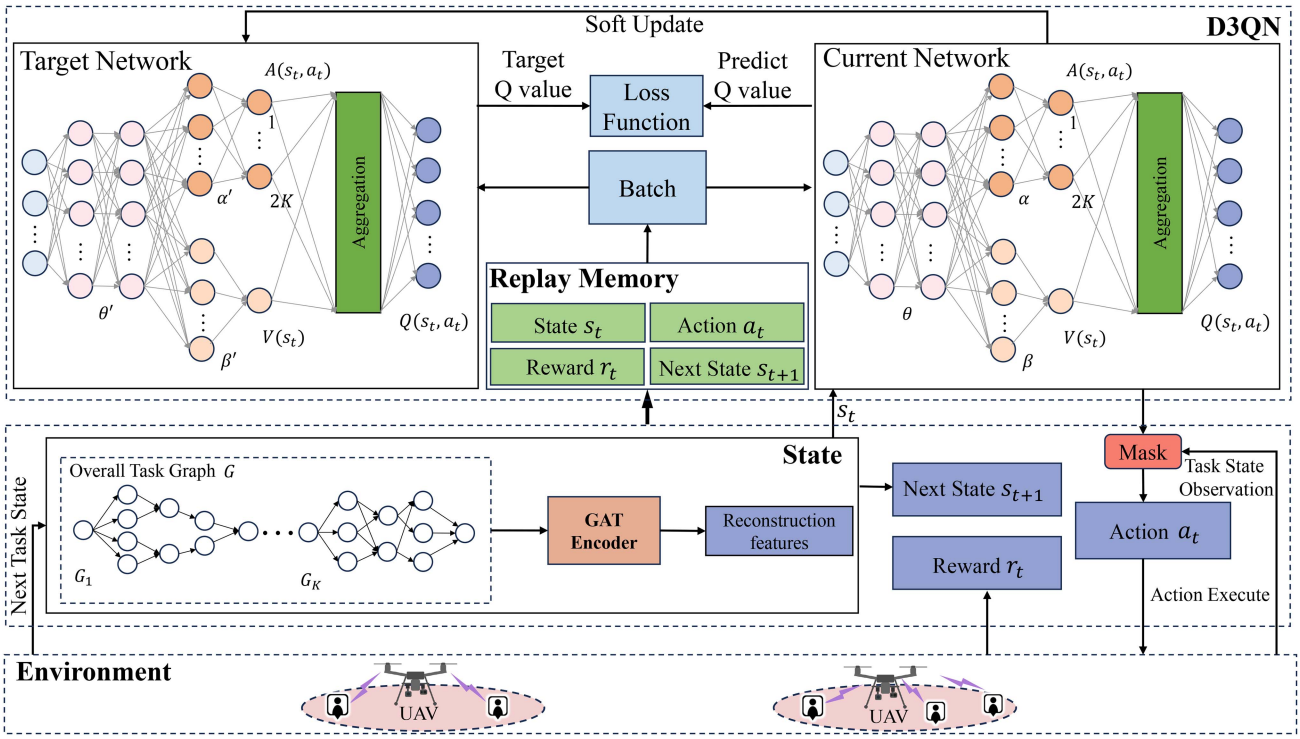



Fig. 3. The architecture of the proposed algorithm.

value less negative and closer to zero. An ideal decision, such as executing a task in parallel without delaying the critical path, results in a reward of $r_t = 0$. Taking Table II as an example, $r_3 = T_3 - T_4 = 0.292 - 0.292 = 0$, which indicates that after executing the action at decision step 3, the end time remains unchanged. Thus, maximizing the rewards means minimizing the end time by ensuring that the increase in latency is minimized at each step. The calculation of $T_t$ is detailed in Algorithm 2.

The structure of D3QN is shown in Fig. 3. It is composed of two deep neural networks (i.e., current network and target network), each consisted of three layers and two streams. $\theta$, $\alpha$ and $\beta$ denote the parameters of the feature layer, the advantage layer, and the state-value layer, respectively, while the corresponding parameters in the target network are denoted as $\theta'$, $\alpha'$, and $\beta'$. The top stream of each network, which consists of a feature layer and advantage layer, is denoted by $A(s_t, a_t; \theta, \alpha)$ and estimates the advantages for each action. While the bottom stream of each network, which consists of a feature layer and state-value layer, is denoted by $V(s_t, ; \theta, \beta)$ and estimates the state-value as a scalar value. The two separate streams are aggregated to produce state-action Q values for all actions, which are calculated as

$$Q(s_t, a_t; \theta, \alpha, \beta) = V(s_t; \theta, \beta) + \Big(A(s_t, a_t; \theta, \alpha) - \mathbb{E}[A(s_t, a_t; \theta, \alpha)]\Big). \tag{50}$$

After that, unavailable actions are filtered out to form a new action set $\mathcal{A}'$. The action $a_t$ is then selected from available actions as follows:

$$a_t = \arg\max_{a \in \mathcal{A}'} Q(s_t, a; \theta, \alpha, \beta). \tag{51}$$

Meanwhile, $\epsilon$-greedy policy is applied to balance exploration and exploitation. After taking the action $a_t$, the agent receives the reward $r_t$ and the next state $s_{t+1}$. The tuple $(s_t, a_t, r_t, s_{t+1})$ from each step is stored in the replay memory. Once the memory capacity reaches a predefined threshold, the parameters of current network are updated, and the corresponding target Q value is calculated by

$$y_t = r_t + \gamma Q(s_{t+1}, a^*; \theta', \alpha', \beta'), \tag{52}$$

where $a^* = \max_{a \in \mathcal{A}''} Q(s_{t+1}, a; \theta, \alpha, \beta)$ and $\gamma$ is discount factor. $\mathcal{A}''$ is the available action set corresponding to the state $s_{t+1}$. Note that the target network with parameters $(\theta', \alpha', \beta')$ is used to evaluate the bootstrapping action in (52), while the Q-network with parameters $(\theta, \alpha, \beta)$ is used for action selection. To update the current network, the TD error is minimized, which is defined as the difference between the target Q value and the predicted Q value. Accordingly, the loss function is defined as

$$\mathcal{L}(\theta, \alpha, \beta) = [y_t - Q(s_t, a_t; \theta, \alpha, \beta)]^2. \tag{53}$$

The parameters $(\theta, \alpha, \beta)$ are updated by applying gradient descent on the loss function. Additionally, the parameters of the target network $(\theta', \alpha', \beta')$ are then soft-updated every $Z$ iterations:

$$\begin{aligned} \theta' &\leftarrow (1 - \iota)\theta' + \iota\theta, \\ \alpha' &\leftarrow (1 - \iota)\alpha' + \iota\alpha, \\ \beta' &\leftarrow (1 - \iota)\beta' + \iota\beta. \end{aligned} \tag{54}$$

where $\iota \in [0, 1]$ is a small constant that determines the update rate. This ensure stability in the learning process by preventing drastic changes in the target values. The main steps of the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

**Algorithm 3:** GAT-Enhanced D3QN Task Offloading.

**Initialize** replay memory to capacity $C$, the discount factor $\gamma$, the learning rate $\iota$, the current network with random parameters $\theta, \alpha, \beta$, and target network with parameters $\theta' \leftarrow \theta, \alpha' \leftarrow \alpha, \beta' \leftarrow \beta$.
**for** episode $= 1$ to max_episode **do**
  Initialize the initial task state $o_0 = \{-1, \ldots, -1\}$, and end time $T_0 = 0$;
  Obtain the state $s_0$ by GATEncoder in (48);
  **for** step $t = 1$ to $N$ **do**
    Filter out the unavailable states from $\mathcal{A}$ by the mask vector, and select $a_t = \arg\max_{a \in \mathcal{A}'} Q(s_t, a; \theta, \alpha, \beta)$ with $\epsilon$-greedy policy;
    Execute action $a_t$ and obtain end time $T_{t+1}$;
    Calculate the reward $r_t$ by (49) and reach the next task state $o_{t+1}$;
    Obtain the next state $s_{t+1}$ by applying GATEncoder to the task state $o_{t+1}$;
    Store transition $(s_t, a_t, r_t, s_{t+1})$ in replay memory;
    Sample $J$ transitions $(s_j, a_j, r_j, s_{j+1})$ from replay memory;
    Calculate the target Q value $y_j$ by (52);
    Update the parameters $\theta, \alpha, \beta$ of current network by the Adam optimizer according to the loss function: $\frac{1}{J}\sum_{j=1}^{J}[y_j - Q(s_j, a_j; \theta, \alpha, \beta)]^2$;
    Update the parameters $\theta', \alpha', \beta'$ of target network every $Z$ iterations according to (54);
    Set $s_t \leftarrow s_{t+1}$.
  **end for**
**end for**

**Algorithm 4:** Joint JSPO and GATD3QN Optimization for Solving Problem (P1).

Initialize UAV deployment and UAV-GU associations using the JSPO algorithm as described in Section IV, setting iteration counter $r \leftarrow 0$;
**repeat**
  Given the current UAV deployment $\{\mathbf{q}_m^r\}$ and UAV-GU associations $\{a_{km}^r\}$, utilize the GAT-enhanced D3QN algorithm to optimize task offloading decisions, obtaining updated offloading decisions $\{x_i^{r+1}\}$;
  Based on these updated offloading decisions, the corresponding data transfer information between tasks is recalculated and fed back into the JSPO algorithm;
  With the updated data transfer information, re-run JSPO to refine UAV deployments $\{\mathbf{q}_m^{r+1}\}$ and UAV-GU associations $\{a_{km}^{r+1}\}$, obtaining the improved configuration for the next iteration;
  Update $r \leftarrow r + 1$;
**until** The relative decrease in objective value of (P1) is less than a threshold.

proposed GAT-enhanced D3QN task offloading algorithm are presented in Algorithm 3.

To further improve the performance of our proposed framework, we develop an iterative optimization process combining the JSPO algorithm and the GAT-enhanced D3QN method. Specifically, after obtaining initial UAV deployments and UAV-GU associations using the JSPO algorithm, we employ the GAT-enhanced D3QN algorithm to generate updated task offloading decisions. These offloading decisions provide updated data requirements between subtasks, reflecting the actual communication needs among GUs and UAVs. Subsequently, the refined data transfer information is fed back into the JSPO algorithm to perform another round of optimization. Through repeated iterations, the framework continuously adjusts and refines UAV positions, UAV-GU associations, and task offloading policies, converging progressively toward an suboptimal solution. The complete iterative optimization procedure is outlined in Algorithm 4.

## VI. Performance Evaluation

### A. Experiment Setup

*1) Simulation parameters setting:* The system includes $K = 10$ GUs and $M = 2$ UAVs acting as aerial edge servers.

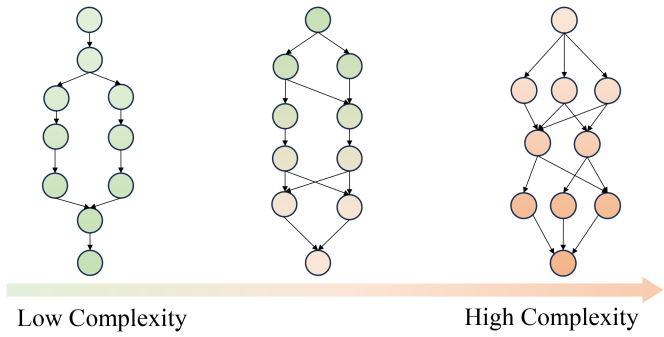



Fig. 4. Different DAGs with 10 nodes under varying complexity levels.

The GUs are randomly distributed within a square area of $1.6 \times 1.6\,\text{km}^2$, while the UAVs hover at a fixed altitude of $H = 100$ meters above the ground. Unless otherwise specified, each GU generates a task consisting of 10 subtasks. The computing capacities of the GUs and UAVs are 2.4 GHz and 4.5 GHz, respectively. The relevant channel parameters are specified as follows: $P_k = 0.1$ W, $\forall k$, $P_m = 0.1$ W, $\forall m$, $B = 1$ MHz, $\sigma^2 = -110$ dBm, $\beta_0 = -50$ dB, $\alpha = 2.2$.

To evaluate our approach comprehensively, we generate diverse task structures using a DAG generator from [36]. Each node in the DAG represents a specific subtask. By adjusting the shape parameter $\alpha$ and regularity parameter $\beta$, we control the complexity of the DAG. A larger $\alpha$ results in a more linear structure, while a higher $\beta$ makes the graph more regular. Consequently, varying these parameters enables us to simulate different complexity levels of task dependencies. As illustrated in Fig. 4, we define complexity based on these parameters, showing that task dependencies become increasingly intricate as $\alpha$ decreases or $\beta$ decreases. Each DAG node and edge is with distinct characteristics. Specifically, the computational load $c_i$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

TABLE III
NEURAL NETWORK ARCHITECTURE AND HYPERPARAMETERS

| Parameter | Value |
|---|---|
| GAT Layers | 2 |
| GAT Hidden Layer dimensions | [64,64] |
| GAT Learning Rate | 0.001 |
| D3QN Layers | 5 |
| D3QN Hidden Layer dimensions | [512,256,128,128,$2K$+1] |
| D3QN Learning Rate | 0.001 |
| Batch Size | 128 |
| Discount Factor $\gamma$ | 0.9 |
| Optimizer | Adam |
| Activation Function | ReLU |

of subtask $i$ is uniformly distributed between $10^8$ and $5 \times 10^8$ cycles. The data transmission size $d_{i,j}$ required to transfer data between subtask $i$ and its successor subtask $j \in \text{suc}(i)$ is uniformly set within the range of 50 to 1000KB.

*2) Neural Network structure:* The detailed neural network structure used in our DRL model consists of five hidden layers. The first four hidden layers contain 512, 256, 128, and 128 neurons, respectively. The final hidden layer serves as the dueling layer and consists of $2^{\sim}K + 1$ neurons: one neuron outputs the state value, while the remaining $2K$ neurons estimate advantage values corresponding to each action. These outputs are combined to produce action-value estimates for the $2K$ available offloading actions, as depicted in Fig. 2. Specific neural network structures and relevant training parameters for both the D3QN and the GAT are summarized in Table III.

### B. Time Complexity Analysis and Convergence Behavior

We have already analyzed the time complexity of the JSPO algorithm in Section IV. For the GAT-enhanced D3QN algorithm, the time complexity of the GAT-enhanced D3QN is influenced by the size of the state-action space, which grows as the number of subtasks increases. In our system model, each task is represented by a DAG, with each subtask corresponding to a node in the graph. The complexity of task offloading decisions depends on the number of nodes and edges in the DAG, as well as the structure of the dependencies between subtasks. To evaluate their performance, we measure the inference time of both GAT and DRL algorithms on a representative hardware platform (Intel Core i7-10700). As illustrated in Fig. 5, the inference time of both GAT and DRL algorithms increases as the number of subtasks increases from 5 to 20. However, even with 20 subtasks, the inference time remains approximately 0.1 seconds, a negligible overhead compared to the typical task execution durations. This result suggests that our proposed method maintains acceptable computational efficiency and is practically feasible for real-world scenarios.

*2) Convergence Behavior:* To evaluate the convergence behavior of our iterative optimization approach, we examine the convergence of the JSPO optimization process and the training convergence of the GAT-enhanced D3QN algorithm. First, the convergence of the JSPO algorithm is illustrated in Fig. 6(a). The result demonstrates the quick convergence of the JSPO

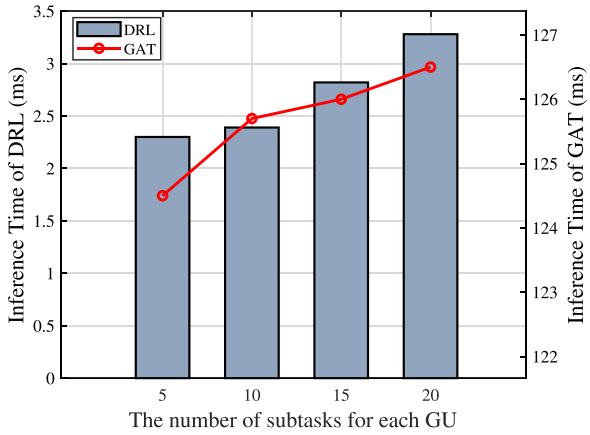
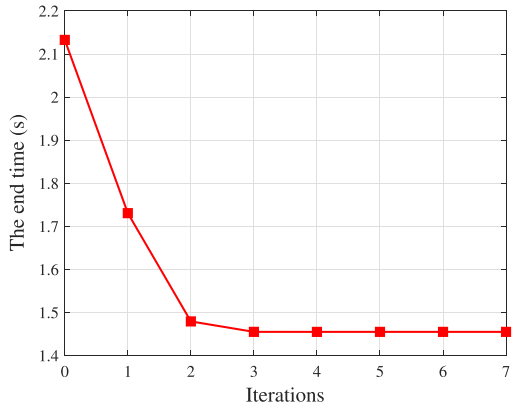
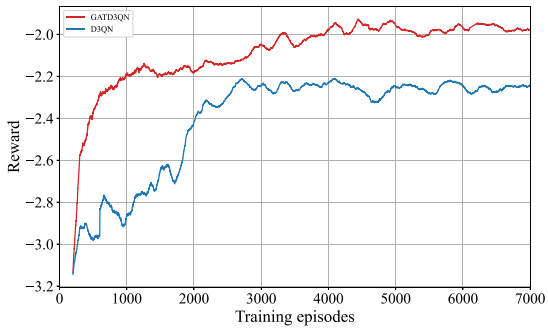



Fig. 5. Impact of the number of subtasks on inference time of GAT and the DRL.




(a) Convergence of the JSPO algorithm.




(b) Convergence of the GAT-enhanced D3QN algorithm.

Fig. 6. Convergence performance of proposed algorithm.

algorithm. The maximum end time drops sharply in the first two iterations and then stabilizes, confirming its efficient convergence. Next, we evaluate the convergence performance of the proposed GAT-enhanced D3QN algorithm. For this purpose, we compare it with a baseline D3QN model that does not utilize GAT for state representation. Fig. 6(b) shows the training curves of both methods. Clearly, the GAT-enhanced D3QN algorithm converges more quickly and stably than the standard D3QN. The GAT-enhanced D3QN algorithm achieves a final reward that is

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

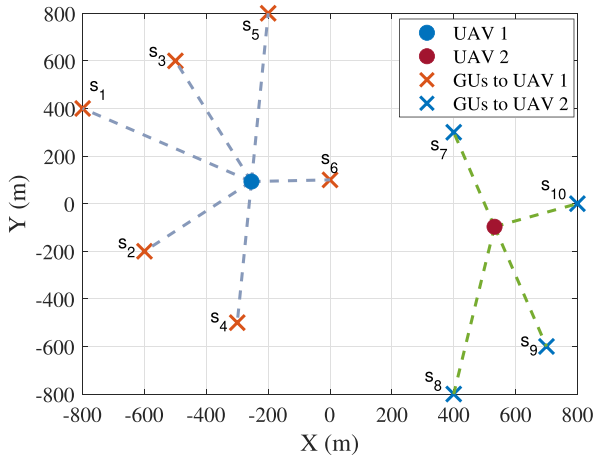



Fig. 7. Optimized deployment and association with JSPO algorithm.

approximately 12% higher than the D3QN, indicating a significantly more efficient offloading policy. Regarding the learning efficiency, a key observation is the rapid initial progress of the GAT-enhanced D3QN. It reaches the D3QN's final performance level after only about 1,000 training episodes. While the D3QN model plateaus at this suboptimal level, our GAT-enhanced agent continues to learn and refine its policy, leveraging the GAT's deeper understanding of the task structure to unlock further performance gains. This ability to avoid premature convergence and find a superior solution is a direct result of the GAT module's capability to capture complex task dependencies. In contrast, the D3QN's limited understanding causes it to get stuck in a local optimum, leading to an inferior policy.

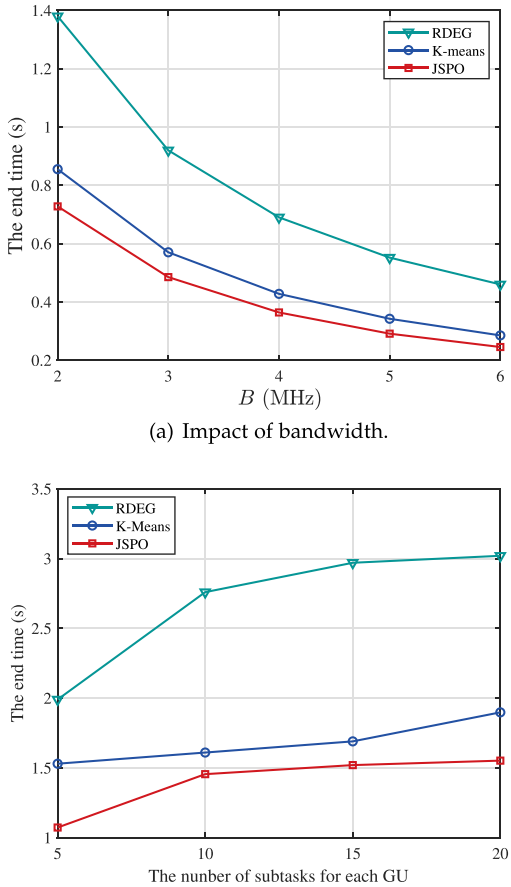
### C. Optimized Deployment and Association

Fig. 7 shows the optimized UAV deployments and UAV-GU associations by JSPO algorithm. This optimization considers the communication requirements of each user, such as data volume and transmission needs. As a result, the associations are adjusted to balance the system load, positioning the UAVs closer to GUs with higher communication costs, which effectively reduces the maximum end time.

To demonstrate the advantages of our solution and ensure fairness, the proposed JSPO and GATD3QN methods are compared separately with other optimization and offloading algorithms. Each method is evaluated against its respective benchmark to highlight its performance improvements. The following sections detail these comparisons, providing insights into the effectiveness of our approach in different scenarios.

For JSPO, the following benchmarks are considered:

- *K-Means* [37]: GUs are clustered into $M$ groups using K-means, with UAVs placed at the cluster centroids and GUs associating with the UAV in their cluster.
- *Random Deployment with Equal GUs (RDEG)* [38]: UAVs are randomly deployed, and each UAV sequentially associates with its nearest GU. The process continues until all UAVs are associated with an approximately equal number of GUs. To mitigate performance variance arising from the UAV deployment, RDEG results are averaged through 100 runs.




(a) Impact of bandwidth.

(b) Impact of number of subtasks for each GU.

Fig. 8. Performance comparison between the JSPO and other algorithms.

Fig. 8(a) compares the end time of the proposed JSPO algorithm with K-Means and RDEG schemes under varying bandwidth $B$. It is observed that JSPO consistently demonstrates the superior performance across all bandwidth levels. While RDEG shows a notable decline in end time as bandwidth $B$ increases, it consistently underperforms compared to K-Means. This is because K-Means generates UAV deployment positions based on GU locations and incorporates global clustering, ensuring a more balanced distribution of resources. However, K-Means does not consider the differing data requirements of GUs, which leads to its inferior performance compared to JSPO.

In Fig. 8(b), it can be seen that the JSPO algorithm consistently demonstrates superior performance in terms of end time when compared to the benchmark methods. Similar to Fig. 8(a), the RDEG method, due to its fundamental assignment strategy, results in elevated end times. The K-Means approach which relies on clustering to deploy UAVs based on GU locations, does not account for the different data demands of each GU. This deficiency leads to its performance inferior to that of JSPO. Moreover, it is observed that as the number of subtasks for each GU increases, the end time does not show a significant increase. This is because the computation time of subtasks is treated as a constant and not included in the process of deployment and association optimization. Therefore, the increase in the number of subtasks does not lead to a significant increase in end time.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

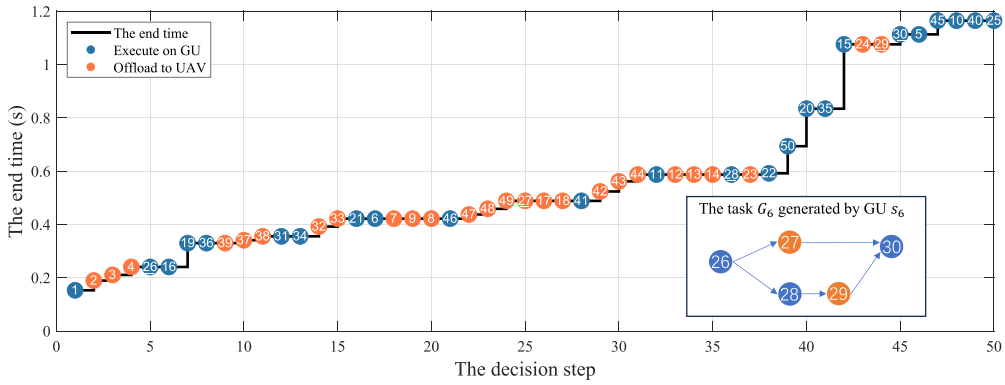



Fig. 9. Optimized task scheduling and offloading decision.

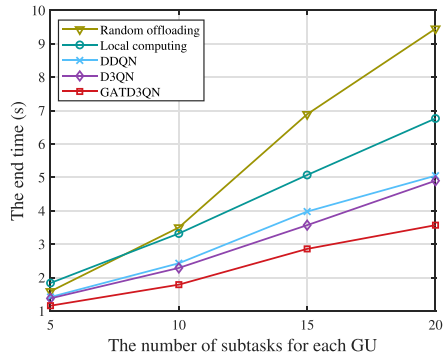
### D. Optimized Task Scheduling and Offloading

For ease of presentation of the task scheduling and offloading process, we show the results where each task consists of 5 subtasks, totaling 50 subtasks for the entire setup. The scheduling and offloading results are illustrated in Fig. 9. It is observed that several subtasks appear on the same horizontal line and the end time does not increase. This is because these subtasks are being processed in parallel, allowing for more computation and transmission without causing an increase in the end time. This parallel processing significantly reduces both computation and communication delays, which is crucial to minimize the end time of tasks. However, towards the end of the graph, we observe a noticeable increase in the end time. This is due to the constraints imposed on the system: the final subtask must be processed locally. Despite the constraint that both the first and last subtasks must be executed locally, most of the remaining subtasks are still processed in parallel without unnecessary waiting or idle computation. This indicates that the proposed algorithm can effectively schedule and offload tasks while exploiting available computation opportunities. As a result, it reduces delays and avoids prolonging the end time. For example, consider task $G_6$ generated by GU $s_6$, which includes subtasks 26 to 30 as shown in Fig. 9. Although the first and last subtasks are executed locally due to the constraint, the execution of intermediate subtasks proceeds in parallel. Notably, none of these subtasks, except the final one, leads to an increase in end time. This illustrates how the algorithm leverages idle periods created by other high-latency tasks, thereby improving parallelism and overall efficiency.

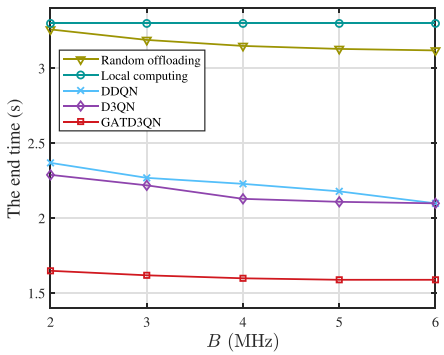
Since our algorithm is based on DDQN, we focus on comparing it with DDQN-related reinforcement learning methods. The following benchmarks are considered:

- *Random offloading* [10]: Subtasks are randomly offloaded to either the associated UAV or local computing in sequence.
- *Local computing* [13]: All subtasks are executed locally without offloading.
- *DDQN* [39]: Offloading decisions are determined using DDQN without GAT-based feature extraction.
- *D3QN* [40]: Offloading decisions are determined using dueling DDQN without GAT-based feature extraction.

To assess the offloading performance, we also consider the impact of two key factors: the number of subtasks for each GU and the bandwidth $B$ available for communication. Fig. 10 present the performance comparisons under varying numbers of subtasks and bandwidth levels, respectively.




(a) Impact of number of subtasks for each GU.




(b) Impact of bandwidth.

Fig. 10. Performance comparison between the GATD3QN and other algorithms.

In Fig. 10(a), it is observed that GATD3QN consistently outperforms the other benchmarks. Specifically, the local computing outperforms random offloading when the number of tasks increases. However, when the number of subtasks is small, random offloading shows slightly better performance than local computing. The reason is that with fewer subtasks, the task dependency structure is relatively simple, and the system experiences minimal delays in both computation and communication, which limits the effectiveness of further optimization. Additionally, random offloading has the opportunity to leverage the high

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

computational power of UAVs. However, as the number of tasks increases, poor scheduling and offloading decisions can lead to significant waiting and delays, which cannot be fully mitigated by the computational advantages of UAVs. The D3QN performs slightly better than DDQN, which can be attributed to the dueling network structure. This structure improves training by separately estimating state values and action advantages, which helps the model make more informed decisions. However, both D3QN and DDQN still face limitations when dealing with complex task dependencies. Since they rely only on simple node-level state representations, they fail to capture both subtask features and the underlying structural relationships within the DAG. As the number of tasks increases and dependencies become more intricate, the advantage of GATD3QN becomes more pronounced.

In Fig. 10(b), we observe that increasing the bandwidth $B$ reduces the end time across all schemes, except for the local execution baseline. Among them, GATD3QN consistently achieves the best performance. Compared to Fig. 8(a), where the end time decreases sharply with increasing bandwidth, the reduction in Fig. 10(b) is more moderate. This is because Fig. 8(a) primarily reflects the cost of data transmission between subtasks, while Fig. 10(b) includes the full process of scheduling and offloading dependent tasks, where computation and communication are more intertwined. GATD3QN initially benefits from increased bandwidth, but the performance gain gradually diminishes. This is because the model effectively schedules tasks in a parallel manner, overlapping computation and transmission where possible. As a result, further increases in bandwidth have a reduced impact on the end time, since many delays are already hidden by parallelism. The random offloading scheme performs better than the local computing baseline, as increased bandwidth helps reduce communication delays and enables partial use of UAV computing resources. However, it still falls short of DRL-based methods due to the absence of coordinated scheduling and offloading. Without a learned strategy, random decisions lead to increased queuing and idle time. Between the DRL-based baselines, D3QN outperforms DDQN, and both benefit from bandwidth increases. Nonetheless, both remain inferior to GATD3QN. Their limited ability to exploit task structures and subtask relationships prevents them from achieving the same level of parallelism. As a result, they suffer from higher computation waiting times and less efficient use of transmission opportunities compared to GATD3QN.

## VII. Conclusion

In this paper, we proposed an efficient multi-UAV-enabled MEC framework for dependent task offloading. We modeled tasks as DAGs to explicitly represent subtask dependencies and jointly optimized UAV deployment, UAV–GU associations, and task offloading decisions. To address this complex optimization problem, we first introduced the JSPO algorithm, integrating both PDD and SCA techniques to optimize initial UAV deployment and UAV–GU associations. Subsequently, we formulated the task offloading problem as an MDP and solved it using a GAT-enhanced D3QN algorithm. The integration of GAT enabled the D3QN framework to effectively capture intricate subtask dependencies within the DAG structure, resulting in improved offloading decisions and enhanced overall system performance. The optimization was conducted iteratively between JSPO and the DRL framework, progressively refining UAV deployment, associations, and offloading decisions. Extensive simulation results confirmed that our approach significantly reduces the end time compared with existing benchmarks.

## Acknowledgment

The authors also thank South West University Big Data Innovation Center for providing the computing resources.

## References

[1] Y. Siriwardhana, P. Porambage, M. Liyanage, and M. Ylianttila, "A survey on mobile augmented reality with 5G mobile edge computing: Architectures, applications, and technical aspects," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1160–1192, Second Quarter 2021.

[2] Z. Zhou, Q. Wu, and X. Chen, "Online orchestration of cross-edge service function chaining for cost-efficient edge computing," *IEEE J. Sel. Areas Commun.*, vol. 37, no. 8, pp. 1866–1880, Aug. 2019.

[3] C. Zhan, H. Hu, Z. Liu, J. Wang, and R. Fan, "Interference-aware online optimization for cellular-connected multiple UAV networks with energy constraints," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13804–13820, Dec. 2024.

[4] H. Han, C. Zhan, J. Lv, and C. Xu, "Energy minimization for cellular-connected aerial edge computing system with binary offloading," *IEEE Internet Thing J.*, vol. 11, no. 6, pp. 9558–9571, Mar. 2024.

[5] Y. Xu, T. Zhang, J. Loo, D. Yang, and L. Xiao, "Completion time minimization for UAV-assisted mobile-edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 70, no. 11, pp. 12253–12259, Nov. 2021.

[6] Q. Wu, M. Cui, G. Zhang, F. Wang, Q. Wu, and X. Chu, "Latency minimization for UAV-enabled URLLC-based mobile edge computing systems," *IEEE Trans. Wireless Commun.*, vol. 23, no. 4, pp. 3298–3311, Apr. 2024.

[7] Y. Wang, Z.-Y. Ru, K. Wang, and P.-Q. Huang, "Joint deployment and task scheduling optimization for large-scale mobile users in multi-UAV-enabled mobile edge computing," *IEEE Trans. Cybern.*, vol. 50, no. 9, pp. 3984–3997, Sep. 2020.

[8] J. Liu, J. Ren, Y. Zhang, X. Peng, Y. Zhang, and Y. Yang, "Efficient dependent task offloading for multiple applications in MEC-cloud system," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2147–2162, Apr. 2023.

[9] M. Gao, R. Shen, L. Shi, W. Qi, J. Li, and Y. Li, "Task partitioning and offloading in DNN-task enabled mobile edge computing networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 4, pp. 2435–2445, Apr. 2023.

[10] B. Xu, Z. Kuang, J. Gao, L. Zhao, and C. Wu, "Joint offloading decision and trajectory design for UAV-enabled edge computing with task dependency," *IEEE Trans. Wireless Commun.*, vol. 22, no. 8, pp. 5043–5055, Aug. 2023.

[11] C. Feng, P. Han, X. Zhang, Q. Zhang, Y. Liu, and L. Guo, "Dependency-aware task reconfiguration and offloading in multi-access edge cloud networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 10, pp. 9271–9288, Oct. 2024.

[12] J. Fang, D. Qu, H. Chen, and Y. Liu, "Dependency-aware dynamic task offloading based on deep reinforcement learning in mobile-edge computing," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 2, pp. 1403–1415, Apr. 2024.

[13] L. Zhao et al., "MESON: A mobility-aware dependent task offloading scheme for urban vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4259–4272, May 2024.

[14] S. Liu et al., "Dependent task scheduling and offloading for minimizing deadline violation ratio in mobile edge computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 2, pp. 538–554, Feb. 2023.

[15] X. Wei, L. Cai, N. Wei, P. Zou, J. Zhang, and S. Subramaniam, "Joint UAV trajectory planning, DAG task scheduling, and service function deployment based on DRL in UAV-empowered edge computing," *IEEE Internet Thing J.*, vol. 10, no. 14, pp. 12826–12838, Jul. 2023.

[16] S. Shen, G. Shen, Z. Dai, K. Zhang, X. Kong, and J. Li, "Asynchronous federated deep-reinforcement-learning-based dependency task offloading for UAV-assisted vehicular networks," *IEEE Internet Thing J.*, vol. 11, no. 19, pp. 31561–31574, Oct. 2024.

[17] F. Khoramnejad, A. Syed, W. S. Kennedy, and M. Erol-Kantarci, "Energy and delay aware general task dependent offloading in UAV-aided smart farms," *IEEE Trans. Netw. Service Manag.*, vol. 21, no. 5, pp. 5033–5048, Oct. 2024.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.

[18] Z. Zhou et al., "Learning-based URLLC-aware task offloading for internet of health things," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 2, pp. 396–410, Feb. 2021.
[19] B. Zhu, K. Chi, J. Liu, K. Yu, and S. Mumtaz, "Efficient offloading for minimizing task computation delay of NOMA-based multiaccess edge computing," *IEEE Trans. Commun.*, vol. 70, no. 5, pp. 3186–3203, May 2022.
[20] W. Chu, X. Jia, Z. Yu, J. C. Lui, and Y. Lin, "Joint service caching, resource allocation and task offloading for MEC-based networks: A multi-layer optimization approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2958–2975, Apr. 2024.
[21] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.
[22] C. Zhan, H. Hu, J. Wang, Z. Liu, and S. Mao, "Tradeoff between age of information and operation time for UAV sensing over multi-cell cellular networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2976–2991, Apr. 2024.
[23] Z. Ning et al., "5G-Enabled UAV-to-community offloading: Joint trajectory design and task scheduling," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3306–3320, Nov. 2021.
[24] X. Jing, F. Liu, C. Masouros, and Y. Zeng, "ISAC from the sky: UAV trajectory design for joint communication and target localization," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 12857–12872, Oct. 2024.
[25] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2322–2358, Fourth Quarter 2017.
[26] Y. Inoue and S.-I. Minato, "An efficient method for indexing all topological orders of a directed graph," in *Proc. 25th Int. Symp. Algorithms Comput.*, 2014, pp. 103–114.
[27] C. Xu, C. Zhan, H. Yang, and L. Xiao, "Pareto-optimal aerial-ground energy minimization for aerial 3D mobile edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 5, pp. 7218–7233, May 2024.
[28] C. Zhan, H. Yan, R. Fan, H. Hu, S. Xu, and J. Yang, "Online energy and interference management for dynamic target tracking with cellular-connected UAV," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5496–5510, Jun. 2025.
[29] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge Univ. Press, 2004.
[30] Q. Shi and M. Hong, "Penalty dual decomposition method for nonsmooth nonconvex optimization–Part I: Algorithms and convergence analysis," *IEEE Trans. Signal Process.*, vol. 68, pp. 4108–4122, 2020.
[31] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018.
[32] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?," in *Proc. Int. Conf. Learn. Representations*, 2022.
[33] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
[34] Y. Zeng, X. Xu, S. Jin, and R. Zhang, "Simultaneous navigation and radio mapping for cellular-connected UAV with deep reinforcement learning," *IEEE Trans. Wireless Commun.*, vol. 20, no. 7, pp. 4205–4220, Jul. 2021.
[35] H. van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[36] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 682–694, Mar. 2014.
[37] H. Guo, X. Zhou, J. Wang, J. Liu, and A. Benslimane, "Intelligent task offloading and resource allocation in digital twin based aerial computing networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3095–3110, Oct. 2023.
[38] J.-H. Kim, M.-C. Lee, and T.-S. Lee, "Generalized UAV deployment for UAV-assisted cellular networks," *IEEE Trans. Wireless Commun.*, vol. 23, no. 7, pp. 7894–7910, Jul. 2024.
[39] X. Chen, S. Hu, C. Yu, Z. Chen, and G. Min, "Real-time offloading for dependent and parallel tasks in cloud-edge environments using deep reinforcement learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 3, pp. 391–404, Mar. 2024.
[40] C. Xu, P. Zhang, H. Yu, and Y. Li, "D3QN-based multi-priority computation offloading for time-sensitive and interference-limited industrial wireless networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 13682–13693, Sep. 2024.



**Cheng Zhan** (Member, IEEE) received the BEng and PhD degrees in computer science from the School of Computer Science, University of Science and Technology of China, Anhui, China, in 2006 and 2011, respectively. From 2009 to 2010, he was a research assistant with the Department of Computer Science, City University of Hong Kong. From 2016 to 2017, he was a visiting scholar with the Department of Electrical and Computer Engineering, National University of Singapore. He is currently an professor with the School of Computer and Information Science, Southwest University, China. His research interests include unmanned aerial vehicle communications, multimedia communications, wireless sensor networks, and network coding. He served as a TPC member for the IEEE ICC, GLOBECOM, WCNC, and UIC.



**Wei Liu** is currently working toward the master degree with the School of Computer and Information Science, Southwest University, Chongqing, China. His research interests include edge computing, computation offloading, and reinforcement learning.



**Kaifeng Song** received the BE degree from the Beijing Institute of Technology, China, in 2022. He is currently working toward the PhD degree with the School of Information and Electronics, Beijing Institute of Technology, China. His research interests include semantic communication and edge intelligence.



**Rongfei Fan** (Member, IEEE) received the BE degree in communication engineering from the Harbin Institute of Technology, Harbin, China, in 2007, and the PhD degree in electrical engineering from the University of Alberta, Edmonton, Alberta, Canada, in 2012. Since 2013, he has been a faculty member with the Beijing Institute of Technology, Beijing, China, where he is currently an associate professor with the School of Cyberspace Science and Technology. His research interests include edge computing, federated learning, resource allocation in wireless networks, and statistical signal processing.



**Jun Liu** (Member, IEEE) received the BEng degree in computer science from Wuhan University, China, in 2002, and the PhD degree in computer science and engineering from the University of Connecticut, USA, in 2013. Currently, he is a professor with the School of Electronic and Information Engineering, Beihang University, China. His major research focuses on underwater networking, synchronization, and localization. He is a member of the IEEE Computer Society.



**Han Hu** (Member, IEEE) received the BE and PhD degrees from the University of Science and Technology of China, China, in 2007 and 2012, respectively. He is currently a professor with the School of Information and Electronics, Beijing Institute of Technology, China. His research interests include multimedia networking, edge intelligence, and space-air-ground integrated network. He received several academic awards, including Best Paper Award of IEEE TCSVT 2019, Best Paper Award of IEEE Multimedia Magazine 2015, Best Paper Award of IEEE Globecom 2013, etc. He served as an associate editor of *IEEE Transactions on Multimedia* and *Ad Hoc Networks*, and a TPC member of Infocom, ACM MM, AAAI, IJCAI, etc.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 13:41:00 UTC from IEEE Xplore. Restrictions apply.